

#DATASET

StudentID	CGPA	IQ	Placement
1	7.2	120	1
2	8.4	110	1
3	6.5	100	0
4	7.8	130	1
...	...	...	...
47	8.2	125	1
48	6.7	95	0
49	7.9	105	1
50	8.5	135	1

#Printing the dataset

```
cgpa package
0 6.89 3.26
1 5.12 1.98
2 7.82 3.25
3 7.42 3.67s
4 6.94 3.57
.. ... ..
195 6.93 2.46
196 5.89 2.57
197 7.21 3.24
198 7.63 3.96
199 6.22 2.33
```

[200 rows x 2 columns]

#Printing the head of the dataset

```
cgpa package
0 6.89 3.26
1 5.12 1.98
2 7.82 3.25
3 7.42 3.67
4 6.94 3.57
```

EXP.NO: 01	BASICS OF STATISTICS
DATE:	

AIM

To perform basic statistical analysis on a dataset using Python by applying functions to compute mean,median, mode, standard deviation, and other descriptive statistics for better understanding of the data.

PROCEDURE:

- Import the required libraries and load the dataset.
- Display basic information and structure of the dataset
- Perform descriptive statistical analysis (mean, median, mode, etc.).
- Use grouping, sorting, and slicing to analyze specific data patterns.

PROGRAM:

#Imporing libraries

```
import pandas as pd

import numpy as np
```

#Importing dataframe

```
df=("/content/placement (2).csv")

df=pd.read_csv (df)
```

#Printing the dataframe

```
print(df)
```

#Printing the head of the dataframe

```
print(df.head())
```

#Printing the tail of the dataframe

```
print(df.tail())
```

#Printing sample of the dataset

```
print(df.sample())
```

Antony Johans

714024247008

#Printing the tail of the dataset

	cgpa	package
195	6.93	2.46
196	5.89	2.57
197	7.21	3.24
198	7.63	3.96
199	6.22	2.33

#Printing sample daaser

	cgpa	package
23	6.19	2.48
19	7.48	2.83
174	7.32	2.99

#Printing the description of the dataset

	cgpa	package
count	200.000000	200.000000
mean	6.990500	2.996050
std	1.069409	0.691644
min	4.260000	1.370000
25%	6.190000	2.487500
50%	6.965000	2.995000
75%	7.737500	3.492500
max	9.580000	4.620000

#Printing the size of the dataframe

400

#Printing the shape of the dataframe

(200, 2)

#Printing the columns of the dataframe

Index(['cgpa', 'package'], dtype='object')

#Printing the mean of the dataframe

cgpa 6.99050
package 2.99605
dtype: float64

#Printing the describe of the dataframe

```
print(df.describe())
```

#Printing the size of the dataframe

```
print(df.size)
```

#Printing the shape of the dataframe

```
print(df.shape)
```

#Printing the columns of the dataframe

```
print(df.columns)
```

#Printing the mean of the dataframe

```
print(df.mean())
```

#Printing the median of the dataframe

```
print(df.median())
```

#Printing the mode of the dataframe

```
print(df.mode())
```

Printing the mean (average) of the last 5 rows of the DataFrame

```
print(df.tail().mean())
```

Sorts the DataFrame 'df' based on the 'cgpa' column in ascending order

```
print(df.sort_values(by='cgpa'))
```

Prints the minimum value from the 'package' column

```
print(df['package'].min())
```

Prints the average (mean) value of the 'cgpa' column

```
print(df['cgpa'].mean())
```

Prints the average (median) value of the 'cgpa' column

```
print(df['cgpa'].median())
```

Prints the average (mode) value of the 'cgpa' column

```
print(df['cgpa'].mode())
```

#Printing the median of the dataframe

```
cgpa    6.99050
package 2.99605
dtype: float64
```

#Printing the mode of the dataframe

```
cgpa package
0 5.84    2.60
1 NaN     2.99
2 NaN     3.08
```

Printing the mean (average) of the last 5 rows of the DataFrame

```
cgpa    6.776
package 2.912
dtype: float64
```

Sorts the DataFrame 'df' based on the 'cgpa' column in ascending order

```
cgpa package
84 4.26    1.53
193 4.57   1.94
88 4.73    1.48
85 4.79    1.48
143 4.85   1.54
.. ...    ...
63 9.16    4.25
51 9.26    3.79
169 9.31   4.37
40 9.38    4.02
50 9.58    4.43
```

[200 rows x 2 columns]

Prints the minimum value from the 'package' column

```
1.37
```

Prints the average (mean) value of the 'cgpa' column

```
6.99050000000001
```

Prints the average (median) value of the 'cgpa' column

```
6.965
```

Prints the average (mode) value of the 'cgpa' column

```
0 5.84
Name: cgpa, dtype: float64
```

PROGRAM &EXECUTION	
CLASS PERFORMNCE	
VIVA	
TOTAL	

RESULT:

This code performs basic statistical analysis on a dataset using Python to compute key descriptive statistics for better data understanding..

#DATASET

StudentID	CGPA	IQ	Placement
1	7.2	120	1
2	8.4	110	1
3	6.5	100	0
4	7.8	130	1
...	...	...	...
47	8.2	125	1
48	6.7	95	0
49	7.9	105	1
50	8.5	135	1

#Checking the missing values

```

Student_ID  CGPA  IQ  Placement
0    False  False  False   False
1    False  False  False   False
2    False  False  False   False
3    False  False  False   False
4    False  False  False   False
..      ...    ...      ...    ...
95    False  False  False   False
96    False  False  False   False
97    False  False  False   False
98    False  False  False   False
99    False  False  False   False

```

```
[100 rows x 4 columns]
```

#Printing the columns of the dataframe

```
Index(['Student_ID', 'CGPA', 'IQ', 'Placement'], dtype='object')
```


EXP.NO: 02**DATA VISUALIZATION****DATE:****AIM:**

To perform data visualization on a given dataset using Python by applying various graphical techniques such as bar chart, histogram, scatter plot, line plot, and pie chart for better understanding and interpretation of data patterns, trends, and relationships.

PROCEDURE:

Import the required libraries and load the dataset

Display basic information and structure of the dataset

Perform descriptive statistical analysis

Use grouping, sorting, and slicing to analyze specific data patterns

Visualize the data using appropriate plots

PROGRAM:**# Importing the required libraries**

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plot
```

Loading the dataset

```
df = ("/content/Placement.csv")
df = pd.read_csv(df)
```

Check for the missing values

```
print(df.isnull())
```

#Prints the column of the dataset

```
print(df.columns)
```

#Prints the description of the dataframe

	Student_ID	CGPA	IQ	Placement
Count	100.000000	100.000000	100.000000	100.000000
Mean	50.500000	5.991000	123.580000	0.500000
Standard Deviation (std)	29.011492	1.143634	39.944198	0.502519
Minimum (min)	1.000000	3.300000	37.000000	0.000000
25% (Q1)	25.750000	5.075000	101.500000	0.000000
50% (Median / Q2)	50.500000	6.000000	127.500000	0.500000
75% (Q3)	75.250000	6.900000	149.000000	1.000000
Maximum (max)	100.000000	8.500000	233.000000	1.000000

#Prints the size of the dataframe

400

#Prints the dimension of the dataframe

2

Displaying specific rows and columns (IQ and CGPA)

	IQ	CGPA
1	106	5.9
2	121	5.3
3	132	7.4
4	142	5.8

Filling missing values in 'Student_ID' column with mean

	Student_ID
0	1
1	2
2	3
3	4
4	5
..	...
95	96
96	97
97	98
98	99
99	100

[100 rows x 1 columns]

#Prints the description of the dataframe

```
print(df.describe())
```

#Prints the size of the dataframe

```
print(df.size)
```

#Prints the dimension of the dataframe

```
print(df.ndim)
```

#Prints the datatype of the dataframe

```
print(df.dtypes)
```

Displaying specific rows and columns (IQ and CGPA)

```
print(df.loc[1:4, ['IQ', 'CGPA']])
```

Filling missing values in 'Student_ID' column with mean

```
print(df[['Student_ID']].fillna(df[['Student_ID']].mean()))
```

Scatter plot between Student_ID and CGPA

```
A=df['Student_ID'].head(20)
B=df['CGPA'].head(20)
plot.scatter(A[0:10],B[0:10])
plot.show()
```

Bar chart between Student_ID and CGPA

```
A = df['Student_ID'].head(20)
B = df['CGPA'].head(20)
plot.bar(A[0:10], B[0:10])
plot.show()
```

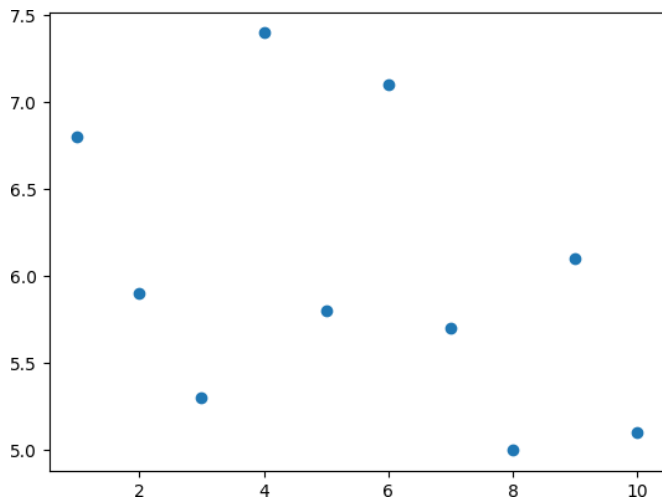
Histogram for Placement

```
A = df['Placement']
plot.hist(A[0:10])
plot.show()
```

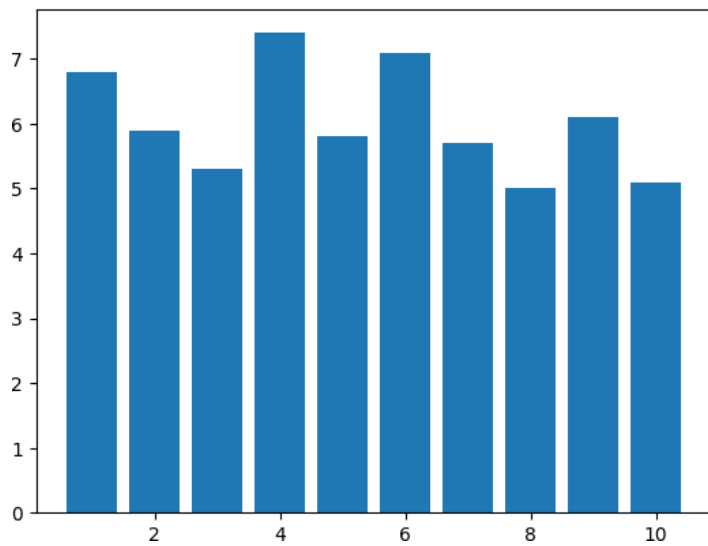
Line plot for CGPA (dotted style)

```
A = df['CGPA']
plot.plot(A[1:10], linestyle='dotted')
plot.show()
```

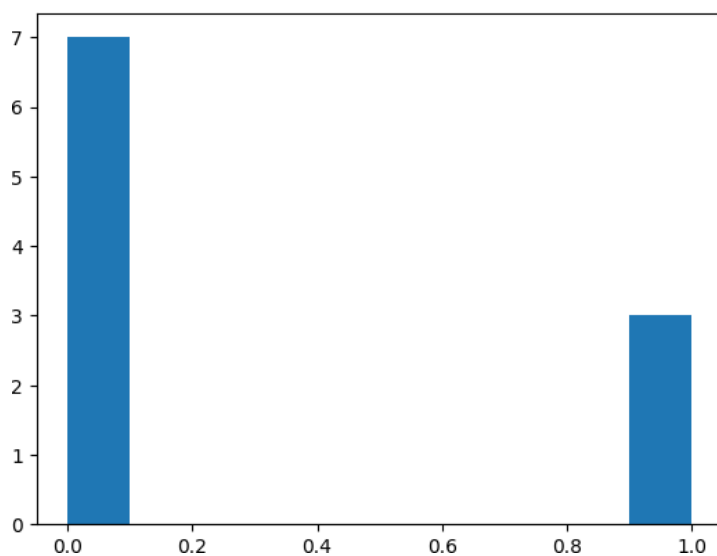
Scatter plot between Student_ID and CGPA



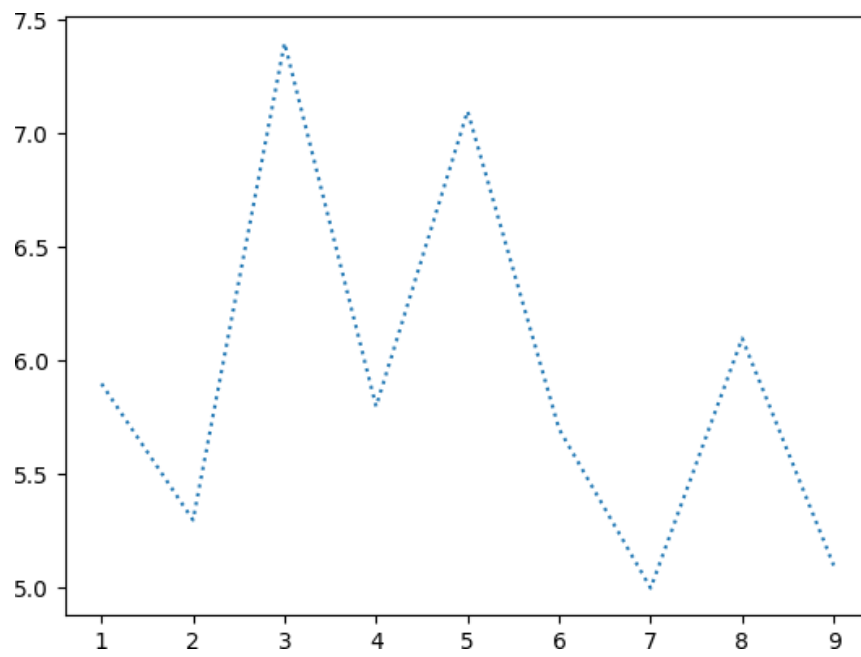
Bar chart between Student_ID and CGPA



Histogram for Placement



Line plot for CGPA (dotted style)



PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

The dataset was analyzed using Python, and statistical measures with visualizations like histograms and bar charts helped understand data trends and distribution.

#DATASET

SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3.0	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
...	...	...	...	...
6.7	3.3	5.7	2.5	Iris-virginica
6.7	3.0	5.2	2.3	Iris-virginica
6.3	2.5	5.0	1.9	Iris-virginica
6.5	3.0	5.2	2.0	Iris-virginica

EXP.NO: 03

K-NEAREST ALGORITHM

DATE:

AIM:

To visualize the Iris dataset using different graphs and apply the K-Nearest Neighbors (KNN) algorithm to classify flower species based on their features.

PROCEDURE:

- Import the required libraries such as pandas, numpy, matplotlib, seaborn, and sklearn.
- Load the Iris dataset and display its basic structure and statistical summary.
- Perform grouping, sorting, and slicing to analyze data patterns and trends.
- Visualize the dataset using bar chart, histogram, scatter plot, line plot, and pie chart.
- Apply the K-Nearest Neighbors (KNN) algorithm for classification and evaluate model accuracy.

PROGRAM:**#Importing Libraries**

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
```

Read the dataset

```
df = pd.read_csv("iris.csv")
```

Select input (X) and output (y)

```
X = df[['sepal.length', 'petal.length']]
y = df['variety']
```

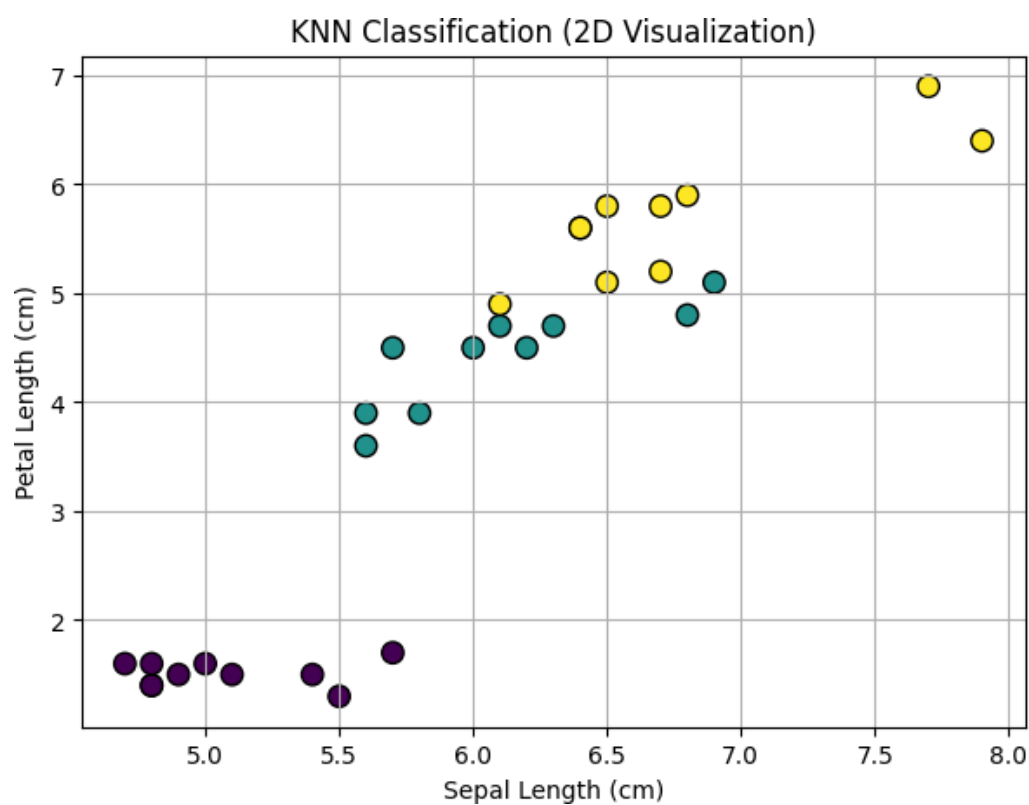
Split the data (80% train, 20% test)

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)
```

#KNN Accuracy of the iris dataset :

KNN Accuracy: 96.67 %

#2D visualization



Scale the data

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Create and train the KNN model

```
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
```

Predict on test data

```
y_pred = knn.predict(X_test_scaled)
```

Check accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("KNN Accuracy:", round(accuracy * 100, 2), "%")
```

2D Visualization

```
plt.figure(figsize=(7,5))
plt.scatter(X_test['sepal.length'], X_test['petal.length'],
c=pd.Categorical(y_pred).codes, cmap='viridis', edgecolor='k', s=80)
```

```
plt.title("KNN Classification (2D Visualization)")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.grid(True)
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

The KNN algorithm was successfully implemented, and the 2D visualization clearly showed the separation of Iris flower classes with high classification accuracy.

#DATASET

CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)
1	9	15	39
2	1	15	81
3	0	16	6
4	3	16	77
...	..	...	...
47	9	71	78
48	2	71	34
49	9	71	83
50	6	71	35

EXP.NO: 04

SIMPLE RERESSION

DATE:

AIM

To implement Simple Linear Regression to predict the dependent variable based on one independent variable using Python.

PROCEDURE:

- Import required libraries and load the dataset.
- Separate independent (X) and dependent (Y) variables.
- Split the data into training and testing sets.
- Train the Linear Regression model using training data.
- Predict and evaluate results using test data.

PROGRAM:**#Imporing libraries**

```
import pandas as pd

import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

import numpy as np
```

Load dataset

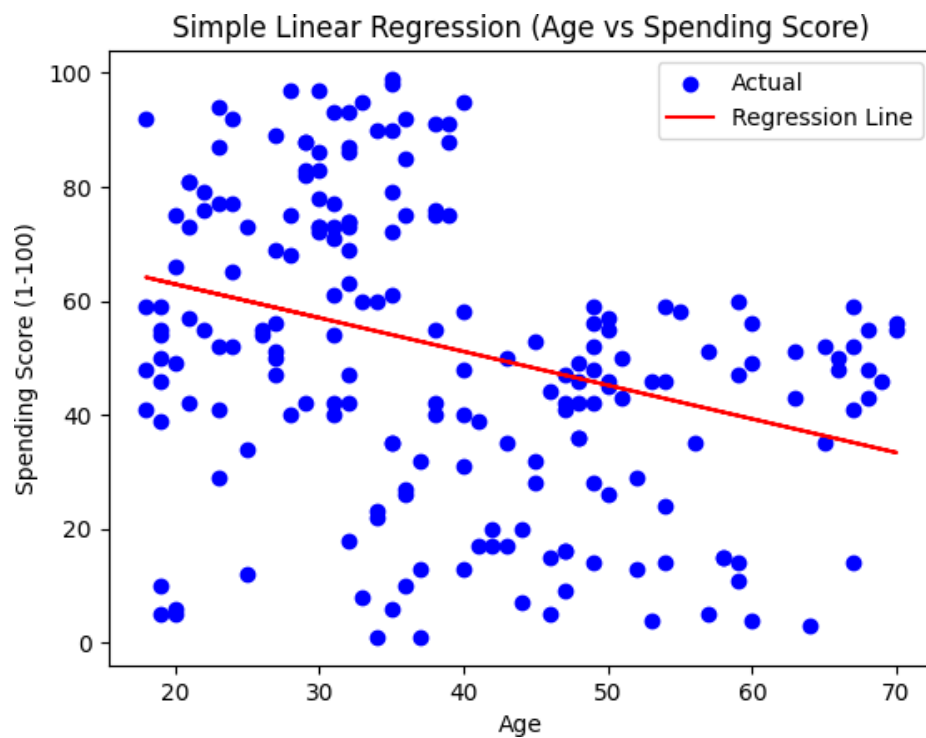
```
df = pd.read_csv("Mall_Customers_Enhanced.csv")
```

#SIMPLE LINEAR REGRESSION (Age vs Spending Score)

```
X_age = df[["Age"]]
y = df["Spending Score (1-100)"]

X_train, X_test, y_train, y_test = train_test_split(X_age, y, test_size=0.2, random_state=42)
model_age = LinearRegression()
model_age.fit(X_train, y_train)
y_pred_age = model_age.predict(X_test)
```

#2D VISUALIZATION



#2D VISUALIZATION

```
plt.scatter(X_age, y, color="blue", label="Actual")
plt.plot(X_age, model_age.predict(X_age), color="red", label="Regression Line")
plt.xlabel("Age")
plt.ylabel("Spending Score (1-100)")
plt.title("Simple Linear Regression (Age vs Spending Score)")
plt.legend()
plt.show()
```

#SIMPLE LINEAR REGRESSION (Annual Income vs Spending Score)

```
X_income = df[["Annual Income (k$)"]]
X_train, X_test, y_train, y_test = train_test_split(X_income, y, test_size=0.2, random_state=42)
model_income = LinearRegression()
model_income.fit(X_train, y_train)
y_pred_income = model_income.predict(X_test)
```

#2D VISUALIZATION

```
plt.scatter(X_income, y, color="green", label="Actual")
plt.plot(X_income, model_income.predict(X_income), color="red", label="Regression Line")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Simple Linear Regression (Income vs Spending Score)")
plt.legend()
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

The **Simple Linear Regression model** was successfully implemented. The relationship between the variables was visualized, and the model predicted the dependent variable accurately based on the given input data.

#DATASET

Hours_Studied	Sleep_Hours	Exam_Score
2	8	55
3	7	60
4	6	65
5	7	70
...	...	...
47	9	88
48	6	66
49	8	92
50	7	85

EXP.NO: 05**LOGICAL REGRESSION****DATE:****AIM:**

To implement Logistic Regression for predicting categorical outcomes based on input data.

PROCEDURE:

- Import required libraries and load the dataset.
- Split the data into training and testing sets.
- Train the regression model Logistic.
- Predict the output for test data.
- Evaluate model performance and display results.

PROGRAM:**# Importing the required libraries**

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
import numpy as np
```

Load the dataset

```
df = pd.read_csv('Student_Marks.csv')
```

Create binary target: 1 if Marks >= 30, else 0

```
df['High_Score'] = (df['Marks'] >= 30).astype(int)
```

Define features (independent variable) and target (dependent variable)

```
X = df[['time_study']]      # Input feature
y = df['High_Score']       # Binary target
```

Split dataset into train and test sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Initialize and train logistic regression model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

Predict on test set

```
y_pred = model.predict(X_test)
```

Evaluate model

```
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

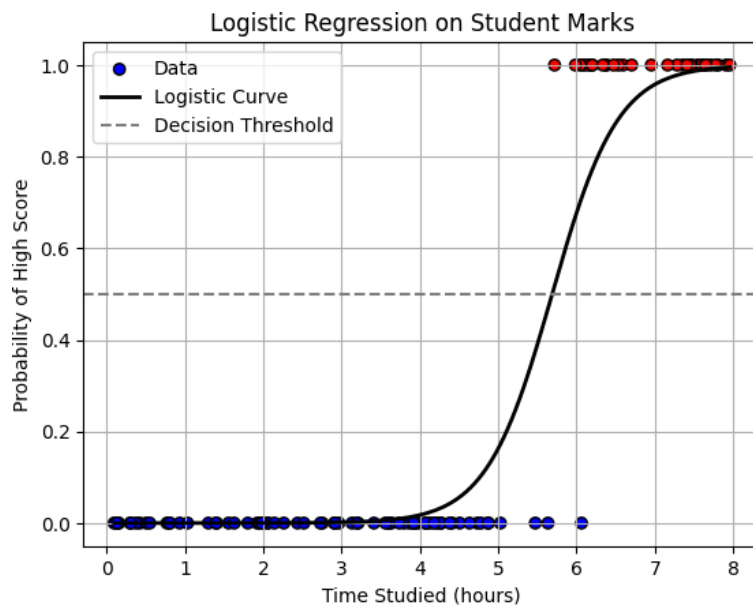
#MODEL EVALUATION

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	1.00	1.00	9
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

#2D VISUALIZATION



Generate smooth input range for plotting the logistic curve

```
X_range = np.linspace(df['time_study'].min(), df['time_study'].max(), 300).reshape(-1, 1)
```

Predict probability for the smooth range

```
probs = model.predict_proba(X_range)[:, 1]
```

Plot actual data points and the logistic regression curve

```
plt.scatter(df['time_study'], df['High_Score'], c=df['High_Score'], cmap='bwr', edgecolors='k', label='Data')
plt.plot(X_range, probs, color='black', linewidth=2, label='Logistic Curve')
plt.axhline(0.5, color='gray', linestyle='--', label='Decision Threshold')
```

Set labels and title

```
plt.xlabel('Time Studied (hours)')
plt.ylabel('Probability of High Score')
plt.title('Logistic Regression on Student Marks')
plt.legend()
plt.grid(True)
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the regression model was implemented successfully, and the output was verified.

#DATASET

mpg cylinders	displacement	horsepower	weight	acceleration	model_year
18.0 8	307.0	130.0	3504	12.0	70
15.0 8	350.0	165.0	3693	11.5	70
18.0 8	318.0	150.0	3436	11.0	70
16.0 8	304.0	150.0	3433	12.0	70
... ..	...	...	...	...	...
32.0 4	135.0	84.0	2295	11.6	82
28.0 4	120.0	79.0	2625	18.6	82
31.0 4	119.0	82.0	2720	19.4	82
27.0 4	140.0	86.0	2790	15.6	82

EXP.NO: 06

POLYNOMIAL REGRESSION

DATE:

AIM:

To implement Polynomial Regression to model the nonlinear relationship between independent and dependent variables.

PROCEDURE:

- Import required libraries and load the dataset.
- Convert input features into polynomial features.
- Split the dataset into training and testing sets.
- Train the Polynomial Regression model using training data.
- Predict and evaluate the model using test data.

PROGRAM:**# Import Libraries**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Load Dataset

```
auto_df = pd.read_csv("auto-mpg.csv")
print("Polynomial Regression on Auto MPG Dataset")
```

Clean the dataset (replace '?' with NaN and drop missing values)

```
auto_df['horsepower'] = auto_df['horsepower'].replace('?', np.nan).astype(float)
auto_df.dropna(inplace=True)
```

Features and Target

```
X = auto_df[['horsepower']]
y = auto_df['mpg']
```

Split dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Transform features into polynomial features (degree=2)

```
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
```

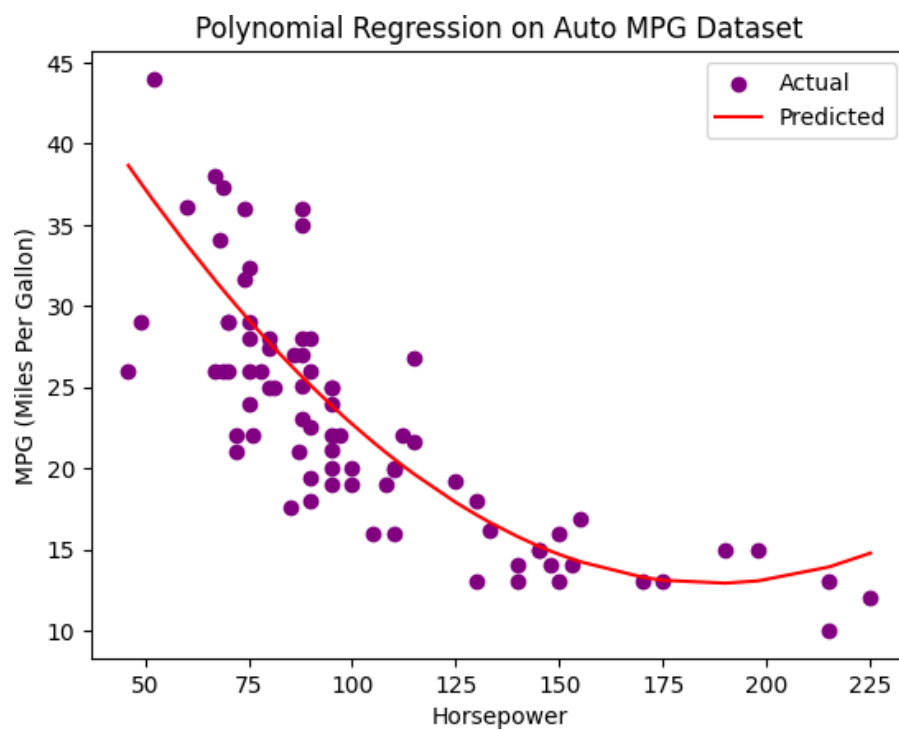
#MODEL EVALUATION

Polynomial Regression on Auto MPG Dataset

Mean Squared Error: 18.4170

R^2 Score: 0.6392

#2D VISUALIZATION



Create and train the Polynomial Regression model

```
PR_model = LinearRegression()
PR_model.fit(X_train_poly, y_train)
```

Make predictions

```
y_pred = PR_model.predict(X_test_poly)
```

Evaluate the model

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f'Mean Squared Error: {mse:.4f}')
print(f'R^2 Score: {r2:.4f}')
```

Plot the results

```
plt.scatter(X_test, y_test, color='purple', label='Actual')
sorted_indices = X_test.squeeze().argsort()
plt.plot(X_test.iloc[sorted_indices], y_pred[sorted_indices], color='red', label='Predicted')
plt.xlabel('Horsepower')
plt.ylabel('MPG (Miles Per Gallon)')
plt.title('Polynomial Regression on Auto MPG Dataset')
plt.legend()
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Thus, the polynomial regression model was implemented successfully, and the output was verified.

#DATASET

ID	X1 (Hours Studied)	X2 (Sleep Hours)	Y (Exam Score)
1	2.5	7.0	58.2
2	3.0	6.5	59.5
3	4.0	6.0	62.1
4	5.0	5.5	65.0
...	...	...	...
44	6.5	6.4	72.0
45	5.9	6.7	70.8
46	4.9	7.0	67.5
47	4.7	6.8	66.2

EXP.NO: 07

MULTI-LINERAR REGRESSION

DATE:

AIM

To implement Multiple Linear Regression using Python to predict Exam Score based on Hours Studied and Sleep Hours.

PROCEDURE:

- Import the required libraries and load the dataset.
- Select input features (Hours Studied, Sleep Hours) and output (Exam Score).
- Split the dataset into training and testing sets.
- Train the Multiple Linear Regression model.
- Predict and evaluate Exam Scores using test data.

PROGRAM:**#Imporing libraries**

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D
```

#Load dataset

```
df = pd.read_csv("multilinear_dataset.csv")
```

Features (independent vars) & Target (dependent var)

```
X = df[['X1 (Hours Studied)', 'X2 (Sleep Hours)']]

y = df['Y (Exam Score)']
```

#OUTPUT

Multiple Linear Regression Results

Coefficients: [3.65790962 0.47492137]

Intercept: 45.044387228705716

Mean Squared Error: 0.9955551840456675

R² Score: 0.9539000914980057

Predictions (rounded): [75.57 69.39 63. 61.77 65.83 72.67 76.68 73.26 69.84 74.5]

Actual Values: [76. 69.9 65.2 62.4 66.5 73.5 77. 73.8 71.5 74.4]

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

Train model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

Predictions

```
y_pred = model.predict(X_test)
```

Evaluation

```
print("\nMultiple Linear Regression Results")
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

Predicted vs Actual

```
print("\nPredictions (rounded):", np.round(y_pred, 2))
print("Actual Values:", y_test.values)
```

3D Visualization (Hours Studied vs Sleep Hours vs Exam Score)

```
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
```

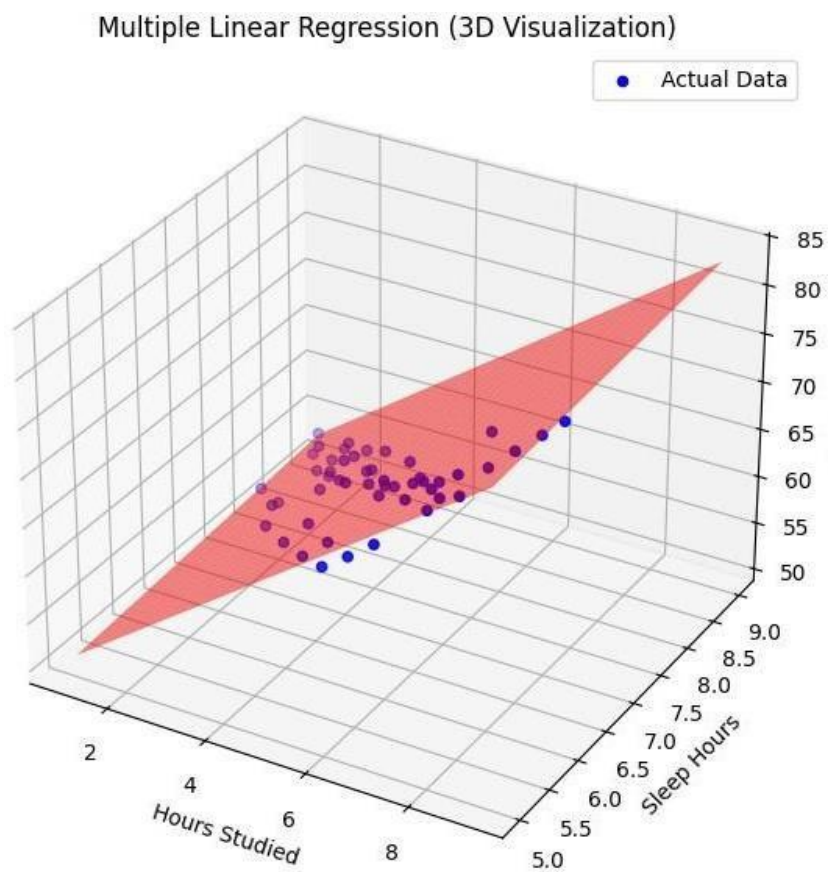
Scatter plot of actual data

```
ax.scatter(df['X1 (Hours Studied)'], df['X2 (Sleep Hours)'], df['Y (Exam Score)'],
    color='blue', label="Actual Data")
```

Create grid for visualization

```
x1_range = np.linspace(df['X1 (Hours Studied)'].min(), df['X1 (Hours Studied)'].max(), 30)
x2_range = np.linspace(df['X2 (Sleep Hours)'].min(), df['X2 (Sleep Hours)'].max(), 30)
```

#2D VISUALIZATION



```
x1_grid, x2_grid = np.meshgrid(x1_range, x2_range)

X_grid = np.c_[x1_grid.ravel(), x2_grid.ravel()]

y_grid_pred = model.predict(X_grid).reshape(x1_grid.shape)
```

Regression plane

```
ax.plot_surface(x1_grid, x2_grid, y_grid_pred, alpha=0.5, color='red')
```

Labels and Title

```
ax.set_xlabel("Hours Studied")

ax.set_ylabel("Sleep Hours")

ax.set_zlabel("Exam Score")

ax.set_title("Multiple Linear Regression (3D Visualization)")

ax.legend()

plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

The Multiple Linear Regression model was successfully implemented and the output is verified.

#DATASET

CustomerID	Age	Annual Income (k\$)	Spending Score (1-100)	Estimated Savings (k\$)
1	19	15	39	8.5
2	21	15	81	22.1
3	20	16	6	5.4
4	23	16	77	21.5
...	...	...	...	...
47	49	71	78	20.6
48	42	71	34	12.7
49	39	71	83	24.2
50	26	71	35	11.4

EXP.NO: 08

MULTINOMIAL REGRESSION

DATE:

AIM:

To implement Multinomial Logistic Regression using Python to classify data into multiple categories based on given features.

PROCEDURE:

- Import required libraries and load the dataset.
- Select input features and output (categorical target).
- Split the dataset into training and testing sets.
- Train the Logistic Regression (multinomial) model.
- Predict and evaluate model accuracy on the test data.

PROGRAM:**#Importing libraries**

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from mpl_toolkits.mplot3d import Axes3D
```

#Load dataset

```
df = pd.read_csv("Mall_Customers_Enhanced.csv")
```

#Create Savings_Category (target variable)

```
def categorize_savings(val):
    if val < 10:
        return "Low"
    elif val < 20:
        return "Medium"
    else:
        return "High"

df['Savings_Category'] = df['Estimated Savings (k$)'].apply(categorize_savings)
```

Features (X) and Target (y)

```
X = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
y_multi = df['Savings_Category']
```

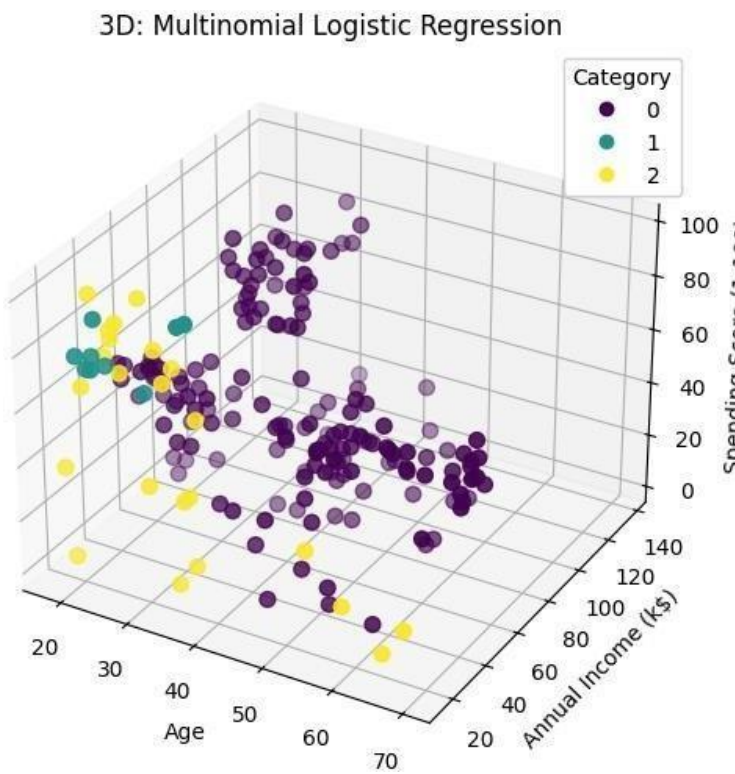
#OUTOUT

Multinomial Logistic Regression Results

Accuracy: 1.0

	precision	recall	f1-score	support
High	1.00	1.00	1.00	36
Low	1.00	1.00	1.00	2
Medium	1.00	1.00	1.00	2
accuracy			1.00	40
macro avg	1.00	1.00	1.00	40
weighted avg	1.00	1.00	1.00	40

#3D VISUALIZATION



#Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split( X, y_multi, test_size=0.2, random_state=42)
```

Multinomial Logistic Regression

```
multi_model = LogisticRegression(multi_class="multinomial", solver="lbfgs", max_iter=1000)
multi_model.fit(X_train, y_train)
y_pred = multi_model.predict(X_test)

print("Multinomial Logistic Regression Results")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

3D Visualization

```
fig = plt.figure(figsize=(7, 6))
ax = fig.add_subplot(111, projection='3d')

y_pred_all = multi_model.predict(X)
scatter = ax.scatter(
    X['Age'], X['Annual Income (k$)'], X['Spending Score (1-100)'],
    c=pd.Categorical(y_pred_all).codes, cmap="viridis", s=50)

ax.set_xlabel("Age")
ax.set_ylabel("Annual Income (k$)")
ax.set_zlabel("Spending Score (1-100)")
ax.set_title("3D: Multinomial Logistic Regression")
legend = ax.legend(*scatter.legend_elements(), title="Category")
ax.add_artist(legend)
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

The Multinomial Logistic Regression model was successfully implemented and the output is verified.

#DATASET

Customer_ID	Age	Gender	Service_Quality	Product_Experience	Complaint_Response	Loyalty_Level	Satisfaction_Level
1	29	Female	Poor	Good	Slow	High	Low
2	39	Male	Poor	Average	Fast	Medium	Low
3	38	Male	Poor	Good	Average	High	Low
4	28	Female	Poor	Good	Slow	Low	Low
...	...	...	...	...	...	...	...
47	44	Male	Poor	Excellent	Average	Medium	High
48	52	Male	Average	Average	Fast	High	Low
49	49	Male	Excellent	Good	Average	High	Very High
50	41	Male	Poor	Excellent	Fast	Medium	Medium

EXP.NO: 09	ORDINAL REGRESSION
DATE:	

AIM:

To implement Ordinal Regression using Python to predict ordered categorical outcomes (e.g., Low, Medium, High) based on given features.

PROCEDURE:

- Import the required libraries and load the dataset.
- Convert the target variable into ordered categories (e.g., 0 = Low, 1 = Medium, 2 = High).
- Split the dataset into training and testing sets.
- Train the Ordinal Regression model on the training data.
- Predict and evaluate the model’s performance using accuracy or confusion matrix.

PROGRAM:

#Importing Libraries

import pandas as pd
import statsmodels.api as sm
from statsmodels.miscmodels.ordinal_model import OrderedModel

Load dataset

df = pd.read_csv("Customer_Feedback_Ordinal.csv")

Define category orders

order_quality = ['Poor', 'Average', 'Good', 'Excellent']
order_product_exp = ['Poor', 'Average', 'Good', 'Excellent']
order_complaint = ['Slow', 'Average', 'Fast', 'Very Fast']
order_loyalty = ['Low', 'Medium', 'High', 'Very High']
order_satisfaction = ['Low', 'Medium', 'High', 'Very High'] # Target categories

Encode categorical variables

df['Service_Quality'] = pd.Categorical(df['Service_Quality'], categories=order_quality, ordered=True).codes
df['Product_Experience'] = pd.Categorical(df['Product_Experience'], categories=order_product_exp, ordered=True).codes
df['Complaint_Response'] = pd.Categorical(df['Complaint_Response'], categories=order_complaint, ordered=True).codes
df['Loyalty_Level'] = pd.Categorical(df['Loyalty_Level'], categories=order_loyalty, ordered=True).codes
df['Satisfaction_Level'] = pd.Categorical(df['Satisfaction_Level'], categories=order_satisfaction, ordered=True).codes

BOWSICKDHARAN P714024247017

#OUTPUT

Model Summary:

OrderedModel Results

```

=====
Dep. Variable:    Satisfaction_Level    Log-Likelihood:    -66.229
Model:            OrderedModel          AIC:              146.5
Method:           Maximum Likelihood    BIC:              159.8
Date:            Thu, 09 Oct 2025
Time:            17:22:01
No. Observations:    50
Df Residuals:        43
Df Model:            4
=====

```

	coef	std err	z	P> z	[0.025	0.975]
Service_Quality	0.1298	0.221	0.587	0.557	-0.304	0.564
Product_Experience	-0.0515	0.265	-0.194	0.846	-0.571	0.468
Complaint_Response	0.0799	0.333	0.240	0.810	-0.573	0.732
Loyalty_Level	0.1913	0.331	0.578	0.563	-0.457	0.840
0/1	-0.4223	0.805	-0.525	0.600	-1.999	1.155
1/2	0.1521	0.240	0.634	0.526	-0.318	0.623
2/3	0.4740	0.251	1.888	0.059	-0.018	0.966

```

=====

```

Predicted survey responses (first 20 rows):

```

Predicted_Response
0  Strongly Disagree
1  Strongly Disagree
2  Strongly Disagree
3  Strongly Disagree
4  Strongly Disagree
5  Strongly Disagree
6  Strongly Disagree
7  Strongly Disagree
8  Strongly Disagree
9  Strongly Disagree
10 Agree
11 Strongly Disagree
12 Agree
13 Strongly Disagree
14 Agree
15 Strongly Disagree
16 Agree
17 Strongly Disagree
18 Strongly Disagree
19 Strongly Disagree

```

Define features and target

```
X = df[['Service_Quality', 'Product_Experience', 'Complaint_Response', 'Loyalty_Level']]
y = df['Satisfaction_Level']
```

Fit ordinal logistic regression

```
model = OrderedModel(y, X, distr='logit')
res = model.fit(method='bfgs', disp=False)
```

--- Model Summary ---

```
print("\nModel Summary:")
print(res.summary())
```

Predict probabilities

```
predicted_probs = res.model.predict(res.params, exog=X)
```

Convert to DataFrame

```
predicted_df = pd.DataFrame(predicted_probs, columns=order_satisfaction)
```

Get predicted category (highest probability)

```
predicted_df['Predicted_Level'] = predicted_df.idxmax(axis=1)
```

Map to survey-style responses

```
mapping = {
    'Low': 'Strongly Disagree',
    'Medium': 'Disagree',
    'High': 'Agree',
    'Very High': 'Strongly Agree'
}

predicted_df['Predicted_Response'] = predicted_df['Predicted_Level'].map(mapping)
```

Show the predicted responses

```
print("\nPredicted survey responses (first 20 rows):")
print(predicted_df[['Predicted_Response']].head(20))
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

The Ordinal Regression model was successfully implemented. It accurately predicted the ordered categories (Low, Medium, High) based on input features.

#DATASET

SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3.0	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
...	...	...	...	...
6.7	3.3	5.7	2.5	Iris-virginica
6.7	3.0	5.2	2.3	Iris-virginica
6.3	2.5	5.0	1.9	Iris-virginica
6.5	3.0	5.2	2.0	Iris-virginica

EXP.NO: 10	SUPPORT VECTOR MACHINE
DATE:	

AIM:

To implement a Support Vector Machine (SVM) algorithm for classifying data into different categories based on features.

PROCEDURE:

- Import the required libraries and load the dataset.
- Split the dataset into training and testing sets.
- Create and train the SVM classifier using the training data.
- Predict the output for the test data.
- Evaluate the model using accuracy or confusion matrix.

PROGRAM:

```
# Import libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

# Load the Iris dataset

iris = datasets.load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

# Let's take only two classes for binary classification (Setosa and Versicolor)

X = X[y != 2]
y = y[y != 2]

# Split dataset into training and testing

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Create the SVM model with linear kernel

svm_model = SVC(kernel='linear')
```

BOWSICKDHARAN P714024247017

#OUTPUT

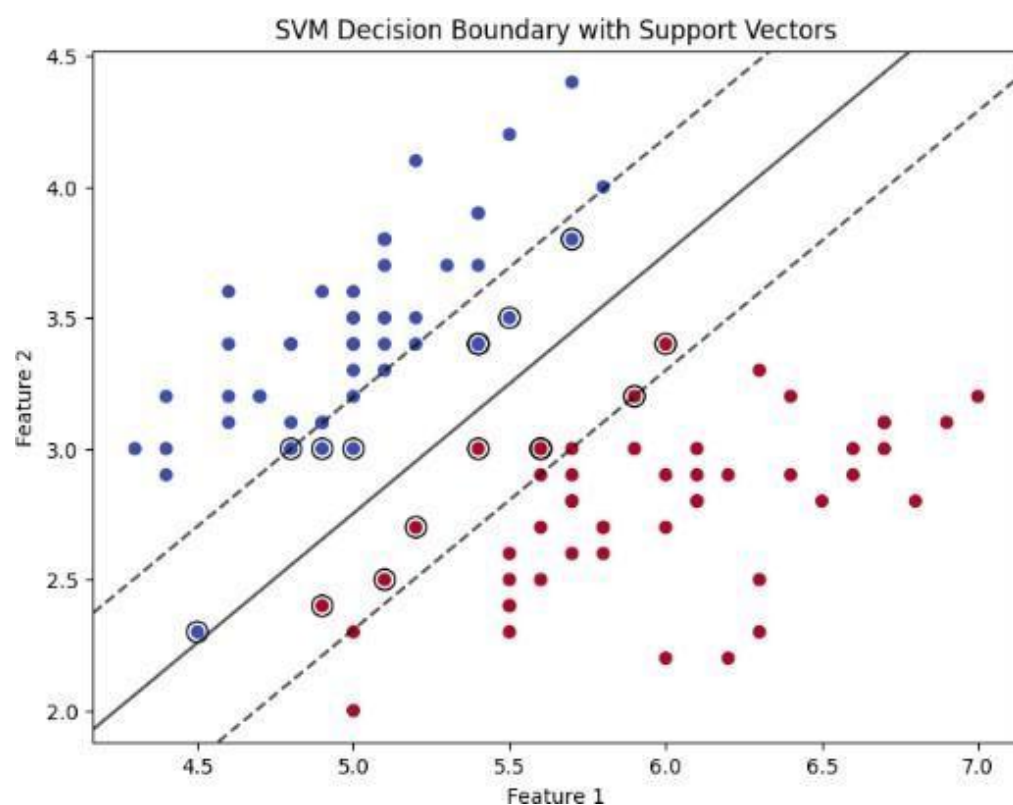
Confusion Matrix:

```
[[17  0]
 [ 0 13]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	17
versicolor	1.00	1.00	1.00	13
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

#2D VISUALIZATION



Train the model

```
svm_model.fit(X_train, y_train)
```

Make predictions

```
y_pred = svm_model.predict(X_test)
```

Evaluate the model

```
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=target_names[:2]))
```

Plotting (only first two features for visualization)

```
def plot_svm(X, y, model):
    plt.figure(figsize=(8, 6))
    X0 = X[:, 0]
    X1 = X[:, 1]
    plt.scatter(X0, X1, c=y, cmap='coolwarm', s=30)

    ax = plt.gca()
    xlim = ax.get_xlim()
    ylim = ax.get_ylim()
```

Create grid to evaluate model

```
xx = np.linspace(xlim[0], xlim[1], 30)
yy = np.linspace(ylim[0], ylim[1], 30)
YY, XX = np.meshgrid(yy, xx)
xy = np.vstack([XX.ravel(), YY.ravel()]).T
Z = model.decision_function(xy).reshape(XX.shape)
```

Plot decision boundary and margins

```
plt.contour(XX, YY, Z, colors='k', levels=[-1, 0, 1], alpha=0.7, linestyles=['--', '-', '--'])
```

Plot support vectors

```
plt.scatter(model.support_vectors_[0], model.support_vectors_[1], s=100,
            linewidth=1, facecolors='none', edgecolors='k')
plt.title("SVM Decision Boundary with Support Vectors")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Only use first two features for plotting

```
plot_svm(X[:, :2], y, SVC(kernel='linear').fit(X[:, :2], y))
```


PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

The Ordinal Regression model was successfully implemented and the output is verified.

#DATASET

<u>CustomerID</u>	Age	Annual Income (k\$)	Spending Score (1-100)
1	9	15	39
2	1	15	81
3	0	16	6
4	3	16	77
...	...	...	...
47	9	71	78
48	2	71	34
49	9	71	83
50	6	71	35

EXP.NO: 11	AGGLOMERATIVE CLUSTERING
DATE:	

AIM:

To implement Agglomerative Hierarchical Clustering using Python to group similar data points into clusters based on distance linkage.

PROCEDURE:

- Import required libraries and load the dataset.
- Compute the distance matrix between all data points.
- Merge the closest pair of clusters based on linkage criteria (single, complete, or average).
- Repeat merging until the desired number of clusters is reached.
- Visualize the clusters using a dendrogram.

PROGRAM:

```
# Import libraries

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
from mpl_toolkits.mplot3d import Axes3D

# Load the dataset

df = pd.read_csv('mall_customer_50.csv')

# Select features for clustering

X = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]

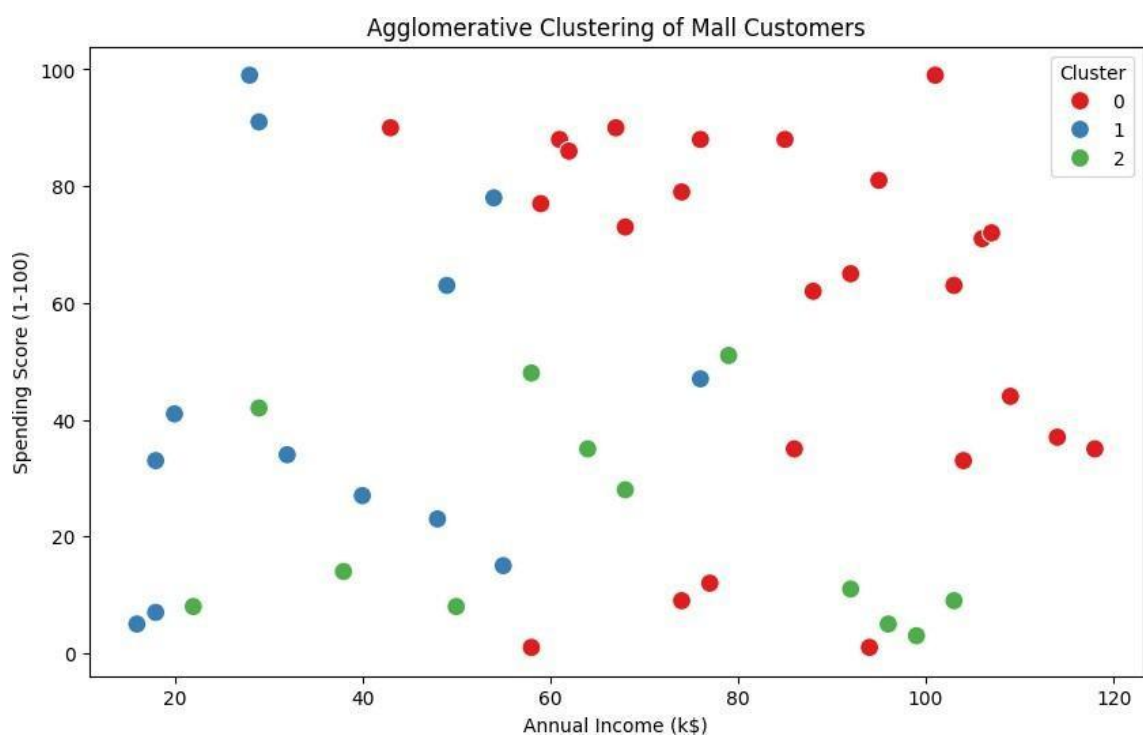
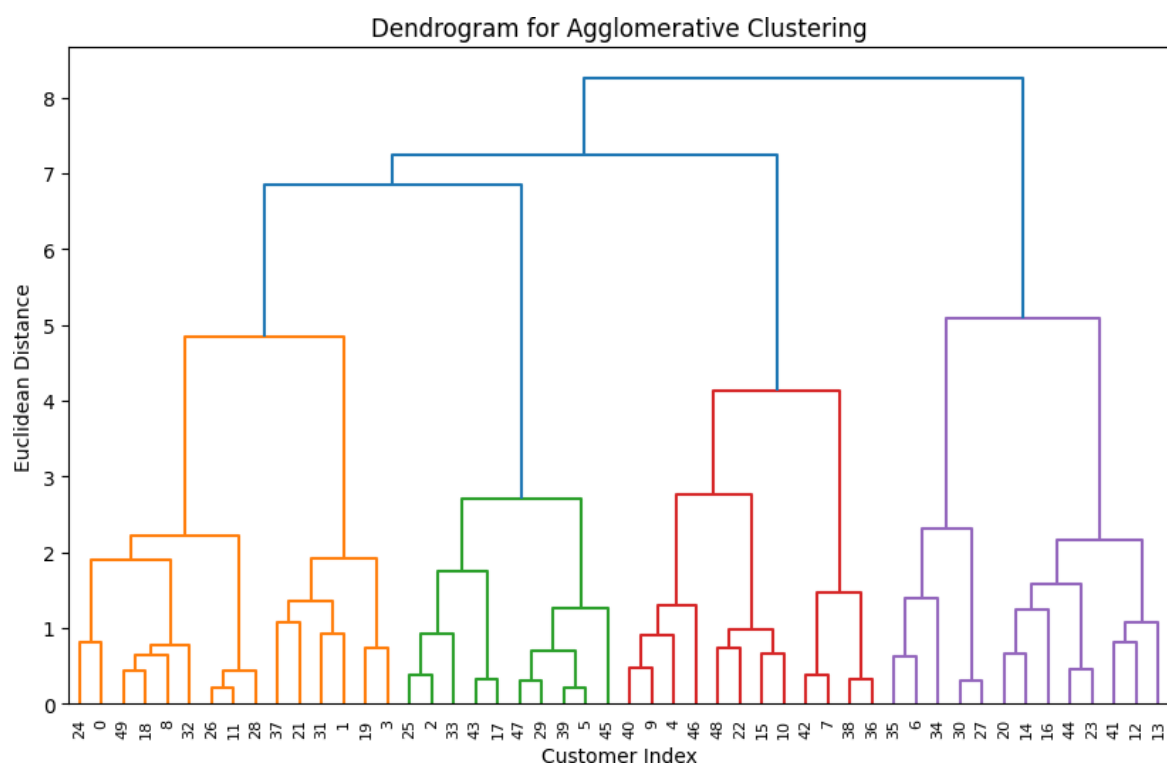
# Standardize the features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

BOWSICKDHARAN P714024247017

#OUTPUT



Create linkage matrix for dendrogram

```
linked = linkage(X_scaled, method='ward')
```

Plot dendrogram

```
plt.figure(figsize=(10, 6))
dendrogram(linked,
            orientation='top',
            distance_sort='descending',
            show_leaf_counts=True)
plt.title('Dendrogram for Agglomerative Clustering')
plt.xlabel('Customer Index')
plt.ylabel('Euclidean Distance')
plt.show()
```

Perform Agglomerative Clustering (compatible with sklearn >=1.2)

```
n_clusters = 3 # Set number of clusters based on dendrogram
cluster_model = AgglomerativeClustering(n_clusters=n_clusters, linkage='ward') # Removed
depreacted 'affinity'
df['Cluster'] = cluster_model.fit_predict(X_scaled)
```

2D Scatter plot visualization

```
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)',
               hue='Cluster', palette='Set1', s=100)
plt.title('Agglomerative Clustering of Mall Customers')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend(title='Cluster')
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMNCE	
VIVA	
TOTAL	

RESULT:

Agglomerative clustering was successfully performed, and the output is verified.

#DATASET

CustomerID	AnnualIncome	SpendingScore
1	35795	9
2	20860	88
3	96820	1
4	74886	8
...	...	...
97	71214	24
98	81228	79
99	68984	59
100	60774	32

EXP.NO: 12

K-MEANS CLUSTERING

DATE:

AIM

To implement **K-Means Clustering** using Python to classify data into k distinct clusters based on similarity.

PROCEDURE:

- ❑ Import the required libraries and load the dataset.
- ❑ Choose the number of clusters (**k**) and initialize centroids.
- ❑ Assign each data point to the nearest centroid.
- ❑ Recalculate centroids based on cluster means.
- ❑ Repeat steps 3–4 until centroids stabilize.

PROGRAM:**# Import libraries**

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

Read dataset from CSV

```
df = pd.read_csv('customer_clustering.csv')
```

Select features for clustering

```
X = df[['AnnualIncome', 'SpendingScore']]
```

Standardize features (important for K-Means)

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

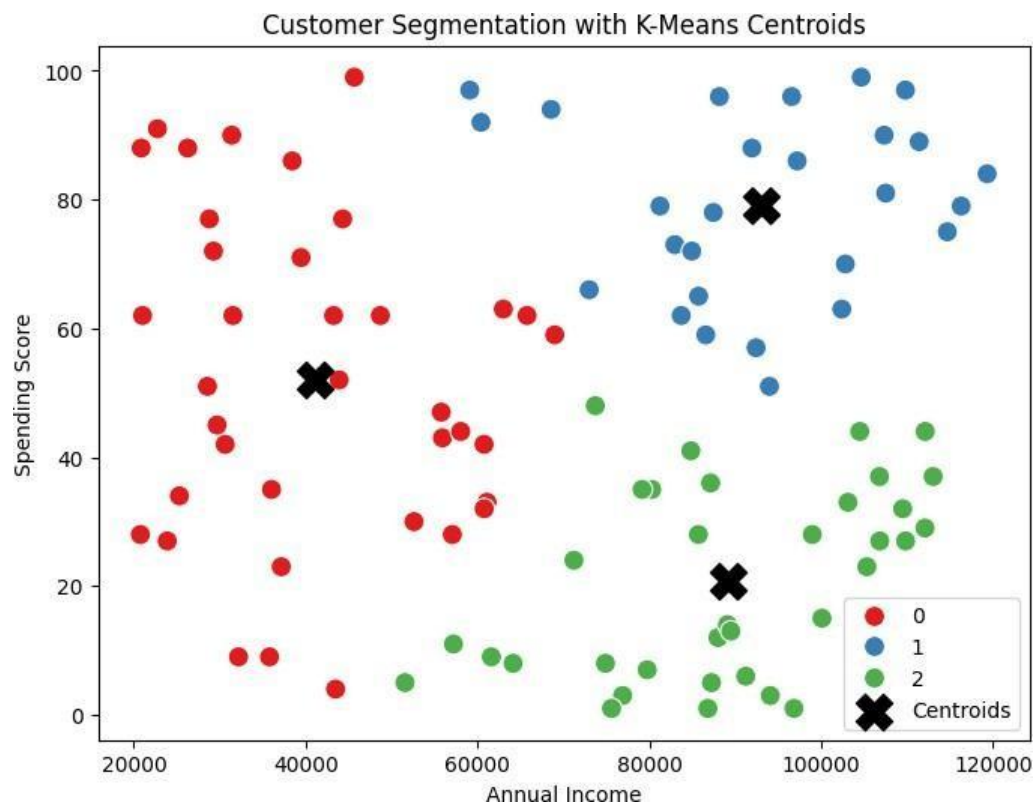
Fit K-Means with chosen number of clusters (e.g., k=3)

```
kmeans = KMeans(n_clusters=3, random_state=42)
df['Cluster'] = kmeans.fit_predict(X_scaled)
```

Get cluster centroids (in scaled space)

```
centroids_scaled = kmeans.cluster_centers_
```

#OUTPUT



Cluster assignments:

	CustomerID	AnnualIncome	SpendingScore	Cluster
0	1	35795	9	0
1	2	20860	88	0
2	3	96820	1	2
3	4	74886	8	2
4	5	26265	88	0
..	...	...	...	...
95	96	58044	44	0
96	97	71214	24	2
97	98	81228	79	1
98	99	68984	59	0
99	100	60774	32	0

[100 rows x 4 columns]

Convert centroids back to original scale

```
centroids = scaler.inverse_transform(centroids_scaled)
```

Visualize clusters with centroids

```
plt.figure(figsize=(8,6))
sns.scatterplot(data=df, x='AnnualIncome', y='SpendingScore', hue='Cluster', palette='Set1',
s=100)
```

Plot centroids

```
plt.scatter(centroids[:,0], centroids[:,1], s=300, c='black', marker='X', label='Centroids')

plt.title('Customer Segmentation with K-Means Centroids')
plt.xlabel('Annual Income')
plt.ylabel('Spending Score')
plt.legend()
plt.show()
```

View cluster assignments

```
print("Cluster assignments:")
print(df[['CustomerID','AnnualIncome','SpendingScore','Cluster']])
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

K-Means clustering was successfully applied, and the data points were grouped into k clusters with distinct centroids.

#DATASET

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
6.1	2.8	4.7	1.2	versicolor
5.7	3.8	1.7	0.3	setosa
7.7	2.6	6.9	2.3	virginica
6.0	2.9	4.5	1.5	versicolor
...	...	...	...	...
5.4	3.9	1.3	0.4	setosa
5.4	3.7	1.5	0.2	setosa
5.5	2.4	3.7	1.0	versicolor
7.2	3.6	6.1	2.5	virginica

EXP.NO: 13	PRINCIPAL COMPONENT ANALYSIS (PCA)
DATE:	

AIM:

To perform K-Means clustering on the Iris dataset before and after applying Principal Component Analysis (PCA) and to visualize the clustering results in 3D, showing how dimensionality reduction affects the data distribution.

PROCEDURE:

- Import dataset and select feature columns for clustering.
- Apply K-Means algorithm on original features to form clusters.
- Standardize the data and reduce dimensions using PCA.
- Apply K-Means again on PCA-transformed features.
- Visualize both clustering results (before and after PCA) using 3D scatter plots.

PROGRAM:

```
# Import libraries

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Load dataset

data = pd.read_csv("iris_50.csv")

# Separate features

X = data[['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']]

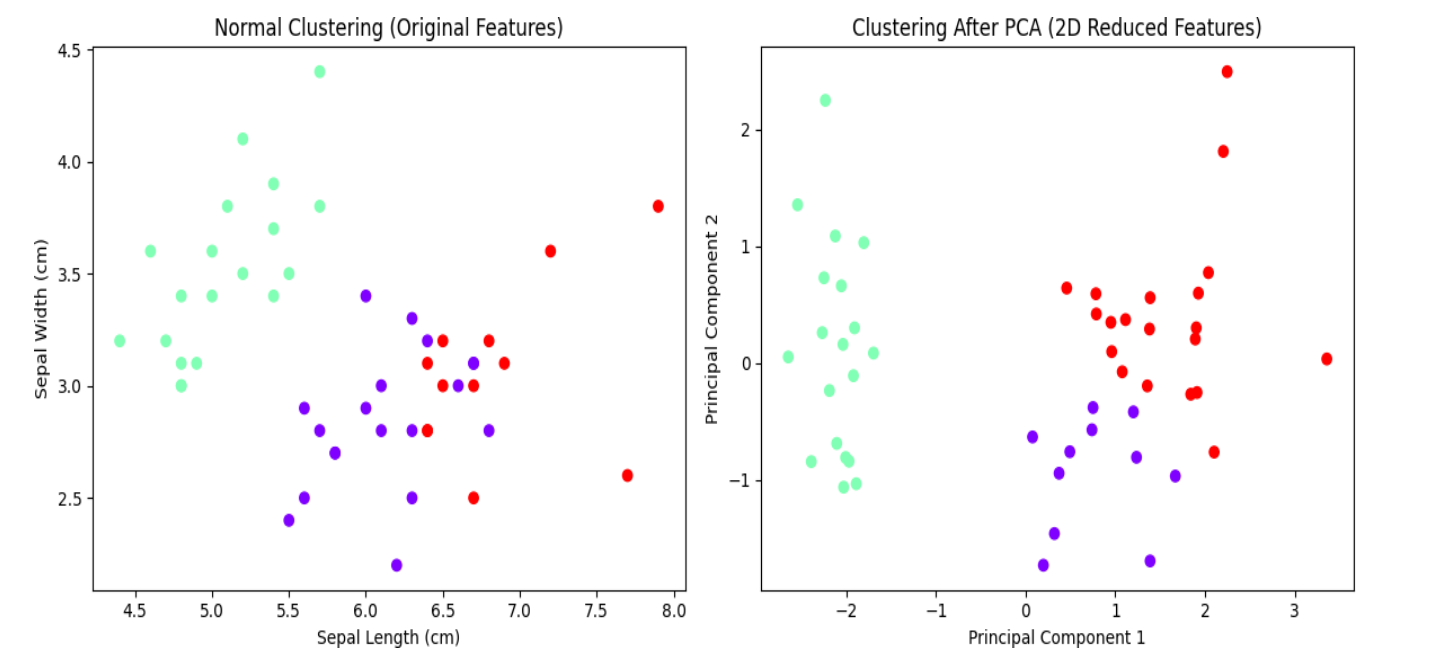
# ----- NORMAL CLUSTERING -----

kmeans_normal = KMeans(n_clusters=3, random_state=42)
data['Cluster_Normal'] = kmeans_normal.fit_predict(X)

# ----- PCA + CLUSTERING -----
# Step 1: Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

BOWSICKDHARAN P714024247017

#OUTPUT



Step 2: Apply PCA

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

Step 3: Cluster on PCA results

```
kmeans_pca = KMeans(n_clusters=3, random_state=42)
data['Cluster_PCA'] = kmeans_pca.fit_predict(X_pca)
```

----- PLOT COMPARISON -----

```
plt.figure(figsize=(12,5))
```

Normal Clustering (using original features)

```
plt.subplot(1,2,1)
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=data['Cluster_Normal'], cmap='rainbow')
plt.title("Normal Clustering (Original Features)")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
```

PCA Clustering (using PCA features)

```
plt.subplot(1,2,2)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=data['Cluster_PCA'], cmap='rainbow')
plt.title("Clustering After PCA (2D Reduced Features)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
```

```
plt.tight_layout()
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

PCA successfully reduced the dataset dimensions while retaining maximum variance.

K-Means clustering after PCA produced clearer and more distinct clusters compared to the original data

#DATASET

Nitrogen	Phosphorus	Potassium	Moisture	Leaf_Temp	Disease_Label
55.52	33.50	60.64	72.05	26.56	0
42.86	32.39	41.37	70.15	27.00	2
36.41	21.32	31.82	72.57	26.19	1
36.78	17.19	33.03	68.04	23.07	1
...	...	...	...	...	...
26.59	18.27	37.37	67.90	26.58	1
44.53	23.48	33.04	76.01	25.18	1
42.77	29.96	46.90	78.59	27.04	0
65.06	36.63	58.47	83.32	22.58	0

EXP.NO: 14	LINEAR DISCRIMINANT ANALYSIS (LDA)
DATE:	

AIM:

To apply LINEAR DISCRIMINANT ANALYSIS (LDA) on the leaf health dataset to reduce multiple features into two discriminant components and visualize class separation between healthy, nutrient-deficient, and diseased plants.

PROCEDURE:

- Load the dataset and separate features (X) and target labels (y).
- Standardize the input features using StandardScaler.
- Apply Linear Discriminant Analysis with 2 components.
- Transform the dataset into lower-dimensional LDA space.
- Plot the data before and after LDA to observe class separation.

PROGRAM:

Step 1: Import required libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.preprocessing import StandardScaler

Step 2: Load the Leaf Health Dataset

df = pd.read_csv("leaf_health_dataset_100.csv")

print("=== Original Dataset (Before LDA) ===")
print(df.head())
print("Shape:", df.shape)

Split into features (X) and target (y)

X = df.drop(columns=["Disease_Label"])
y = df["Disease_Label"]
features = X.columns

Step 3: Standardize the data

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

BOWSICKDHARAN P

714024247017

#OUTPUT

=== Original Dataset (Before LDA) ===

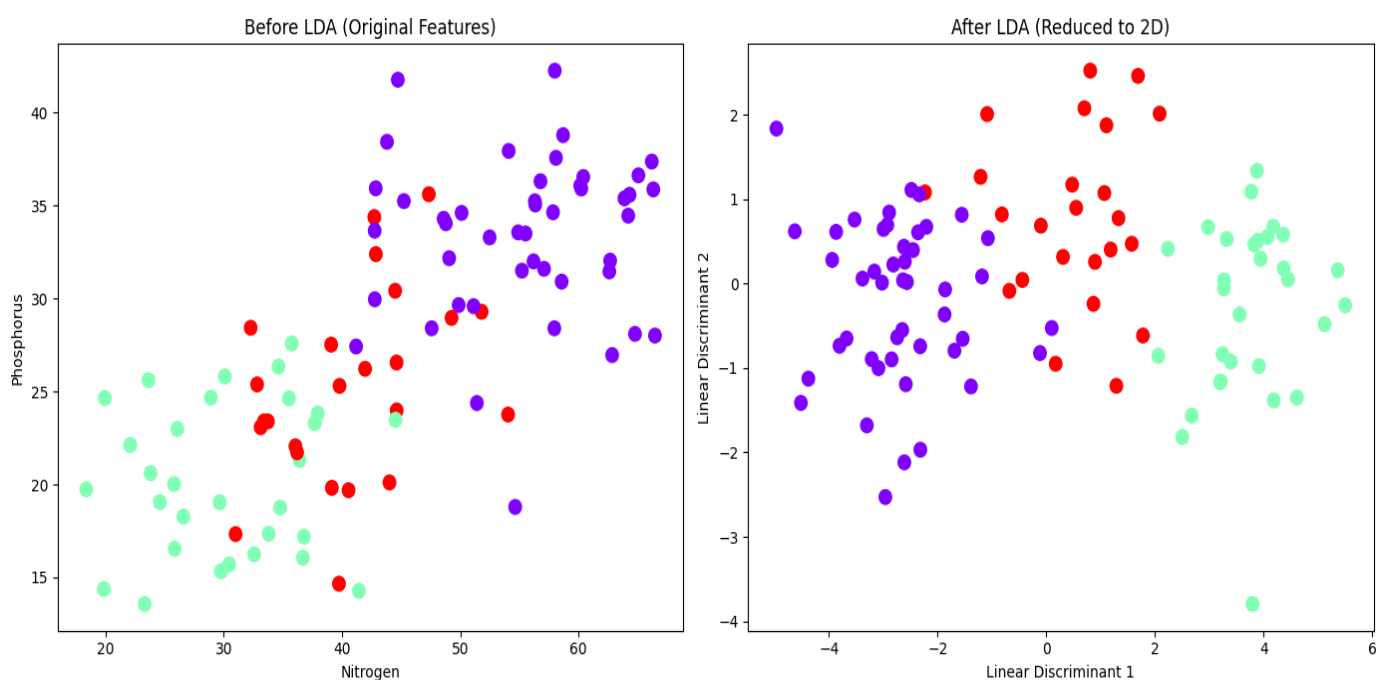
	Nitrogen	Phosphorus	Potassium	Moisture	Leaf_Temp	Disease_Label
0	55.52	33.50	60.64	72.05	26.56	0
1	42.86	32.39	41.37	70.15	27.00	2
2	36.41	21.32	31.82	72.57	26.19	1
3	36.78	17.19	33.03	68.04	23.07	1
4	56.78	36.31	60.04	79.06	24.17	0

Shape: (100, 6)

=== Transformed Dataset (After LDA) ===

	LD1	LD2
0	-2.359204	0.609840
1	0.868954	-0.238171
2	3.398180	-0.923663
3	4.185394	-1.384509
4	-2.584283	-1.191203

#2D-VISUALIZATION



Step 4: Apply LDA to reduce to 2 components

```
lda = LDA(n_components=2)
X_lda = lda.fit_transform(X_scaled, y)

print("\n=== Transformed Dataset (After LDA) ===")
df_lda = pd.DataFrame(X_lda, columns=["LD1", "LD2"])
print(df_lda.head())
```

Step 5: Visualize Before and After LDA

```
plt.figure(figsize=(14,6))
```

Before LDA (First two original features: Nitrogen vs Phosphorus)

```
plt.subplot(1,2,1)
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=y, cmap='rainbow', s=80)
plt.xlabel(features[0])
plt.ylabel(features[1])
plt.title("Before LDA (Original Features)")
```

After LDA (LD1 vs LD2)

```
plt.subplot(1,2,2)
plt.scatter(df_lda["LD1"], df_lda["LD2"], c=y, cmap='rainbow', s=80)
plt.xlabel("Linear Discriminant 1")
plt.ylabel("Linear Discriminant 2")
plt.title("After LDA (Reduced to 2D)")

plt.tight_layout()
plt.show()
```

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

Linear Discriminant Analysis was applied successfully, and the dataset was reduced to two discriminant components. The classes were clearly separated in the transformed space, showing that LDA enhances class discrimination effectively.

#DATASET

Rainfall_mm	Soil_pH	Temperature_C	Fertilizer_kg	CropYield_tph
120	6.2	25	90	3.1
135	6.5	26	88	3.3
140	6.8	27	92	3.5
150	6.9	28	95	3.6
⋮	⋮	⋮	⋮	⋮
145	6.8	27	93	3.4
150	7.0	28	96	3.6
155	7.1	29	100	3.8
160	7.2	30	102	3.9

EXP.NO: 15

INDEPENDENT COMPONENT ANALYSIS (ICA)

DATE:

AIM :

To apply Independent Component Analysis (ICA) on an agriculture dataset and reduce the dimensionality of features (Rainfall, Soil pH, Temperature, Fertilizer) into two independent components, and visualize the data before and after feature extraction.

PROCEDURE:

- Load the agriculture dataset and separate input features (X) and target (y).
- Standardize X using StandardScaler to normalize values.
- Apply FastICA with n_components = 2.
- Transform the standardized data to obtain independent components (IC1, IC2).
- Visualize and analyze IC1 vs IC2 to observe independent feature patterns.

PROGRAM:**# Step 1: Import required libraries**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import FastICA
from sklearn.preprocessing import StandardScaler
```

Step 2: Load Agriculture Dataset

```
df = pd.read_csv("agriculture_data.csv")

print("=== Original Agriculture Dataset (Before ICA) ===")
print(df.head())
print("Shape:", df.shape)
```

Split into features (X) and target (CropYield)

```
X = df.drop(columns=["CropYield_tph"]) # Independent variables
y = df["CropYield_tph"] # Target (just for color visualization)
```

Step 3: Standardize the data

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

#OUTPUT

=== Original Agriculture Dataset (Before ICA) ===

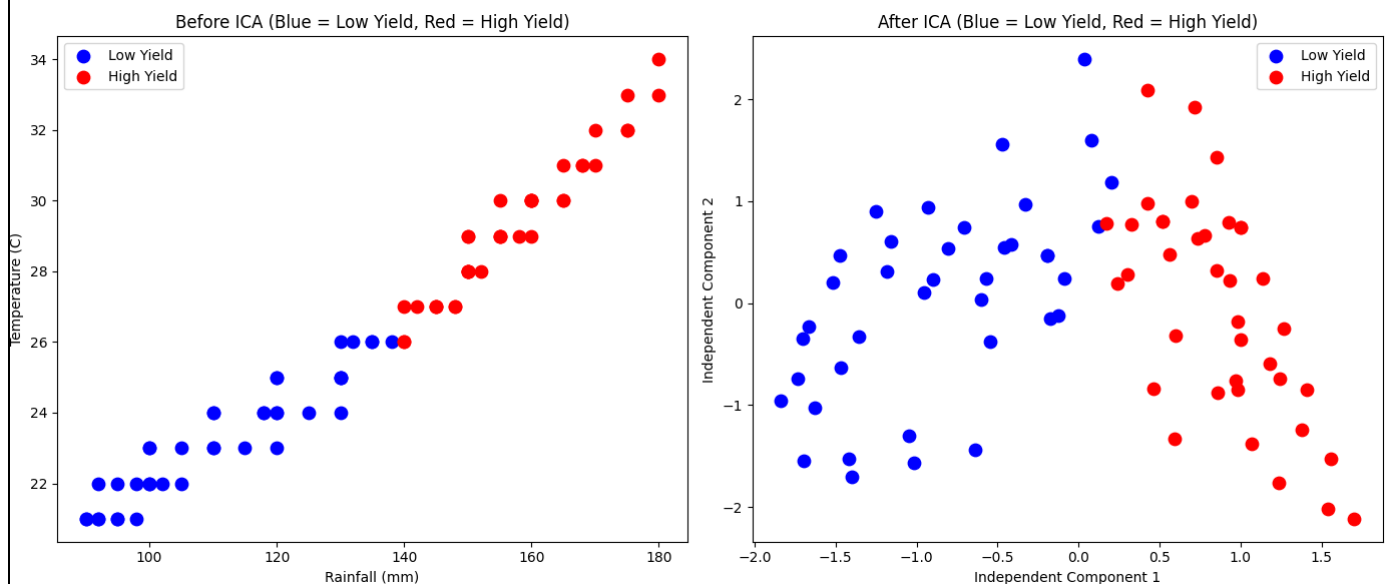
	Rainfall_mm	Soil_pH	Temperature_C	Fertilizer_kg	CropYield_tph
0	120	6.2	25	90	3.1
1	135	6.5	26	88	3.3
2	140	6.8	27	92	3.5
3	150	6.9	28	95	3.6
4	160	7.1	29	98	3.9

Shape: (77, 5)

=== Transformed Dataset (After ICA) ===

	IC1	IC2
0	-0.634992	-1.434978
1	-0.088113	0.237620
2	0.326655	0.773224
3	0.560915	0.480924
4	0.927161	0.796694

#2D-VISUALIZATION



Step 4: Apply ICA and reduce dimensions to 2

```
ica = FastICA(n_components=2, random_state=42)
X_ica = ica.fit_transform(X_scaled)

print("\n=== Transformed Dataset (After ICA) ===")
df_ica = pd.DataFrame(X_ica, columns=["IC1", "IC2"])
print(df_ica.head())
```

Convert yield into 2 groups for red & blue

```
y_binary = (y > y.median()).astype(int) # High yield = 1, Low yield = 0
```

Color map: 0 = Blue, 1 = Red

```
colors = ['blue', 'red']
```

Step 5: Visualization before and after ICA

```
plt.figure(figsize=(14,6))
```

Before ICA (Rainfall vs Temperature)

```
plt.subplot(1,2,1)
for c in [0, 1]:
    plt.scatter(df[y_binary == c]["Rainfall_mm"],
                df[y_binary == c]["Temperature_C"],
                color=colors[c], s=80,
                label="High Yield" if c == 1 else "Low Yield")
plt.xlabel("Rainfall (mm)")
plt.ylabel("Temperature (C)")
plt.title("Before ICA (Blue = Low Yield, Red = High Yield)")
plt.legend()
```

After ICA (Reduced to 2D)

```
plt.subplot(1,2,2)
for c in [0, 1]:
    plt.scatter(df_ica[y_binary == c]["IC1"],
                df_ica[y_binary == c]["IC2"],
                color=colors[c], s=80,
                label="High Yield" if c == 1 else "Low Yield")
plt.xlabel("Independent Component 1")
plt.ylabel("Independent Component 2")
plt.title("After ICA (Blue = Low Yield, Red = High Yield)")
plt.legend()

plt.tight_layout()
plt.show()
```


-

PROGRAM & EXECUTION	
CLASS PERFORMANCE	
VIVA	
TOTAL	

RESULT:

ICA was successfully applied to the agriculture dataset and the original 4 features were reduced to 2 independent components. The data showed clear separation patterns before and after ICA. Thus, ICA effectively extracted independent hidden factors influencing crop yield.